Design Pattern Report: Observer Pattern

Group 52
Nguyễn Đình Minh Huy (Student ID: 25125083)

July 20, 2026

Contents

1 Real-world Problem **2**

2 Naive Solution (Without Design Pattern) **2**

3 Problems with the Naive Solution **3**

4 Introduction to the Observer Pattern **3**

5 General Class Diagram **4**

6 Class Diagram for the Bank Account Problem **4**

7 Implementation of the Observer Pattern **4**

8 Pros and Cons **6**
8.1 Pros 6
8.2 Cons 6

9 Other Real-world Applications **7**
9.1 Web Development 7
9.2 Mobile Development 7

10 Quiz **7**

11 References **8**

1 Real-world Problem

Consider a modern financial core banking system where transactions are performed on a `BankAccount`. When a deposit occurs, several peripheral systems need to be notified of the event immediately to execute their processes:

- `UI`: Updates the account balance display on the user's screen.
- `Email System`: Sends a transaction alert email to the customer.
- `Logger`: Writes transaction details to the security audit trail.
- `Fraud AI`: Scans transaction metadata to detect suspicious patterns.
- `Mobile App`: Delivers a push notification to the client's phone.
- `Analytics`: Records transaction metrics for business intelligence.

A customer or administrator must be able to register, enable, or disable these notification channels dynamically. The core challenge is establishing a system where a bank account can broadcast transaction updates to an arbitrary and dynamic set of observer systems without knowing their concrete classes or internal notification mechanisms.

2 Naive Solution (Without Design Pattern)

In a naive implementation, the `BankAccount` class holds references to concrete notification system objects directly. Below is the C++ representation of this approach:

```cpp
// Refer to Observer/src/naive.cpp for full implementation details
class NaiveBankAccount {
private:
    double balance;
    UI ui;
    EmailSystem email;
    Logger logger;
    FraudDetector fraudDetector;
    MobileApp mobileApp;
    Analytics analytics;

public:
    NaiveBankAccount(double initialBalance) : balance(initialBalance)
   {}

    void deposit(double amount) {
        balance += amount;

        // Direct, coupled method invocations on concrete systems
        ui.update(balance);
        email.sendEmail(amount);
        logger.log(amount, balance);
        fraudDetector.scan(amount);
        mobileApp.pushNotification(amount);
        analytics.trackActivity("Deposit", amount);
    }
};
```

Listing 1: Naive Implementation of Bank Account Notifications

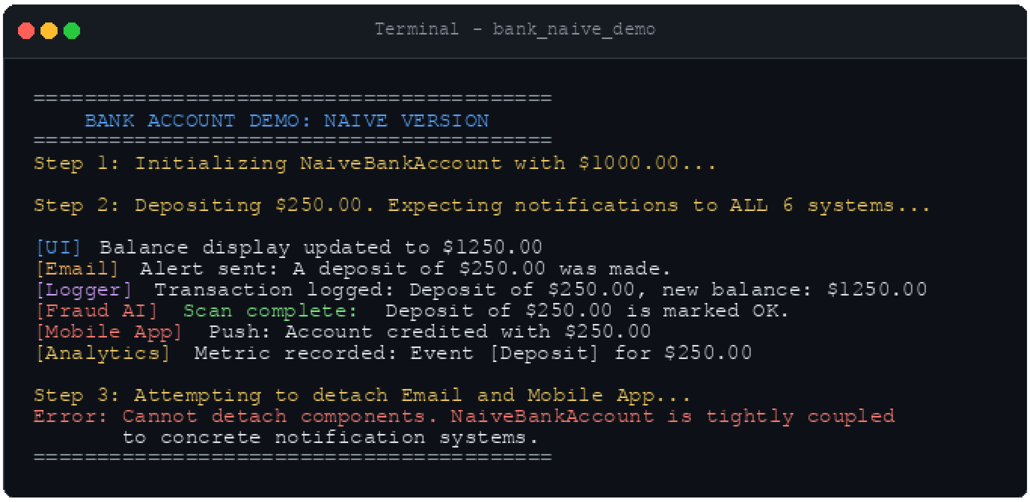

Figure 1: Console Output of the Naive Bank Account Implementation

3 Problems with the Naive Solution

The naive approach exhibits several major architectural flaws:

1. **Tight Coupling:** The `NaiveBankAccount` is tightly coupled to concrete classes like `UI`, `EmailSystem`, `Logger`, `FraudDetector`, `MobileApp`, and `Analytics`. It must know their names, structures, and exactly which update method to call on each.

2. **Violation of the Open-Closed Principle (OCP):** If we want to introduce a new system (such as a compliance reporting system or a credit score monitoring engine), we must modify the core `NaiveBankAccount` class. This requires adding a new member field and updating the notification logic inside the `deposit` method.

3. **Violation of the Single Responsibility Principle (SRP):** The `BankAccount` class should focus solely on financial ledger operations (managing the balance and transactions). However, it is also burdened with orchestrating emails, logs, mobile push notifications, and security analysis.

4 Introduction to the Observer Pattern

The **Observer Pattern** is a behavioral design pattern that defines a one-to-many dependency between objects. When one object (the **Subject** or **Publisher**) changes state, all its dependents (the **Observers** or **Subscribers**) are notified and updated automatically.

Key concepts:

- **Subject:** Knows its observers and provides an interface to attach or detach them.
- **Observer:** Defines an updating interface for objects that should be notified.
- **ConcreteSubject:** Stores state of interest to ConcreteObservers and sends notifications.
- **ConcreteObserver:** Implements the Observer updating interface to keep its state consistent with the subject.

5 General Class Diagram

In the general Observer pattern structure:

- A `Subject` maintains a list of references to `Observer` objects.
- The `Subject` exposes interfaces: `attach(Observer)`, `detach(Observer)`, and `notify()`.
- `Observer` exposes the `update()` method interface.

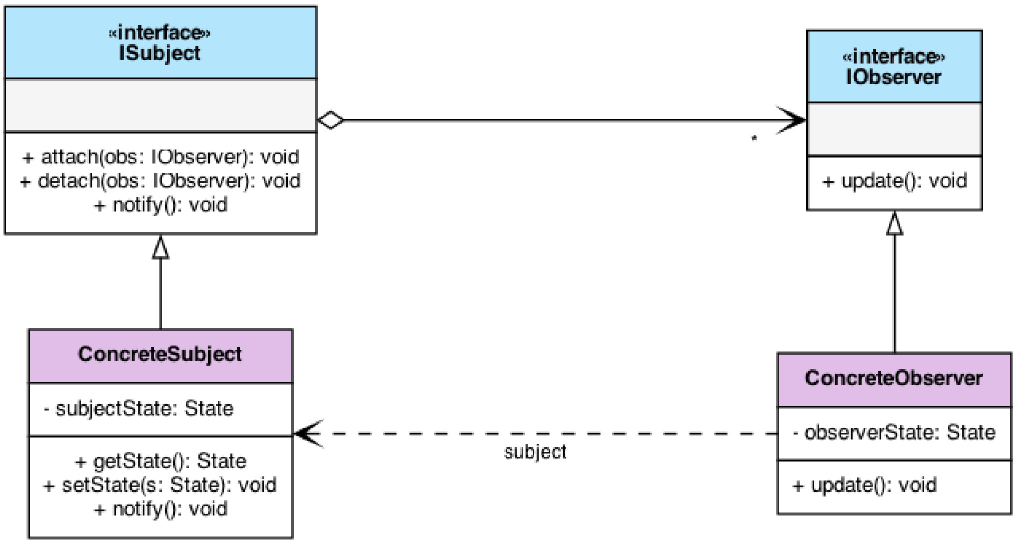

Figure 2: General Observer Design Pattern UML Diagram

6 Class Diagram for the Bank Account Problem

Applying the pattern to our problem:

- `BankAccount` acts as the ConcreteSubject (or subject coordinator).
- `AccountObserver` acts as the Observer interface.
- `UIObserver`, `EmailObserver`, `LoggerObserver`, `FraudAIObserver`, `MobileAppObserver`, and `AnalyticsObserver` act as the ConcreteObservers.

7 Implementation of the Observer Pattern

By decoupling the classes through abstract interfaces, our refactored implementation allows adding or removing subscriber systems at runtime without modifying the core subject.

```
// Refer to Observer/src/pattern.cpp for full implementation details
class AccountObserver {
public:
    virtual ~AccountObserver() = default;
```

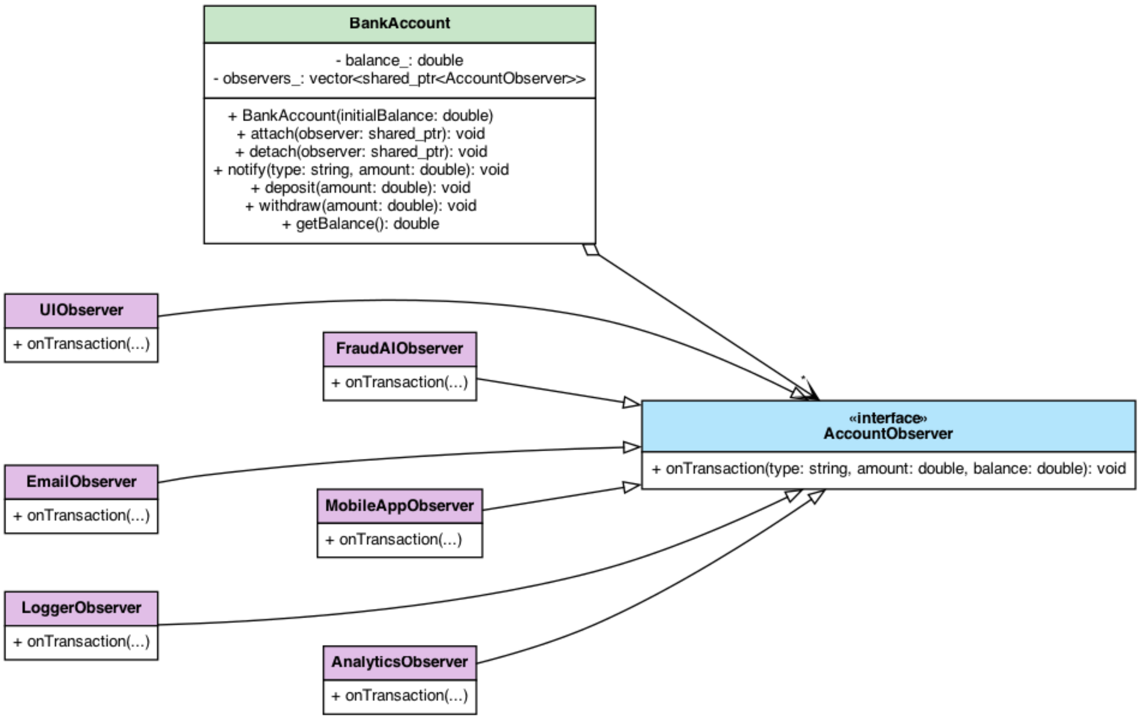

Figure 3: UML Class Diagram for Bank Account Notification System

```cpp
    virtual void onTransaction(const std::string& type, double amount, double currentBalance) = 0;
};

class BankAccount {
private:
    double balance;
    std::vector<std::shared_ptr<AccountObserver>> observers;

public:
    BankAccount(double initialBalance) : balance(initialBalance) {}

    void attach(std::shared_ptr<AccountObserver> obs) { observers.push_back(obs); }
    void detach(std::shared_ptr<AccountObserver> obs) { /* Remove from list */ }
    void notifyObservers(const std::string& type, double amount) {
        for (const auto& obs : observers) {
            obs->onTransaction(type, amount, balance);
        }
    }

    void deposit(double amount) {
        balance += amount;
        notifyObservers("Deposit", amount);
    }
};
```

Listing 2: Refactored Observer Implementation

```
Terminal - bank_observer_pattern_demo

========================================
    BANK ACCOUNT DEMO: PATTERN VERSION
========================================
=== RUNNING PATTERN DEMO (With Observer Pattern) ===
Step 1: Initializing BankAccount with $1000.00...

Step 2: Attaching observers...
 - UIObserver (updates screen balance display)
 - EmailObserver (sends transaction notification emails)
 - LoggerObserver (writes to security/audit logs)
 - FraudAIObserver (scans transaction amounts, warns if > $10,000)
 - MobileAppObserver (sends push notification)
 - AnalyticsObserver (records activity metrics)

Step 3: Depositing $250.00. Expecting notifications to ALL 6 systems...

[BankAccount]  Deposited $250.00. New balance: $1250.00
[UI Observer]  Balance display updated to $1250.00
[Email Observer]  Alert sent: A deposit of $250.00 was made.
[Logger Observer]  Transaction logged: Deposit of $250.00, new balance: $1250.00
[Fraud AI]  Scan complete:  Deposit of $250.00 is marked OK.
[Mobile App]  Push: Account credited with $250.00
[Analytics]  Metric recorded: Event [Deposit] for $250.00

Step 4: Dynamically detaching Email and Mobile App observers...
 (Customer disabled Email alerts and Mobile pushes in settings)

Step 5: Depositing $12000.00. Expecting notifications to remaining systems...
 (No Email, No Mobile push. Fraud AI alert should trigger for large amount)

[BankAccount]  Deposited $12000.00. New balance: $13250.00
[UI Observer]  Balance display updated to $13250.00
[Logger Observer]  Transaction logged: Deposit of $12000.00, new balance: $13250.00
[Fraud AI]  WARNING:  Suspicious large activity detected: Deposit of $12000.00
[Analytics]  Metric recorded: Event [Deposit] for $12000.00

Step 6: Customer re-enables Mobile App notifications in settings...
 (Attaching MobileAppObserver back to BankAccount)

Step 7: Withdrawing $500.00. Expecting notifications to UI, Logger, Fraud AI...

[BankAccount]  Withdrew $500.00. New balance: $12750.00
[UI Observer]  Balance display updated to $12750.00
[Logger Observer]  Transaction logged: Withdrawal of $500.00, new balance: $12750.00
[Fraud AI]  Scan complete:  Withdrawal of $500.00 is marked OK.
[Mobile App]  Push: Account debited with $500.00
[Analytics]  Metric recorded: Event [Withdrawal] for $500.00

Step 8: Attempting to withdraw $15000.00...
 (Expecting Insufficient Funds failure alerts to remaining observers)

[BankAccount]  Rejected: Insufficient funds for withdrawal of $15000.00
[UI Observer]  Balance display updated to $12750.00
[Logger Observer]  Transaction logged: FailedWithdrawal of $15000.00, new balance: $12750.00
[Fraud AI]  WARNING:  Suspicious large activity detected: FailedWithdrawal of $15000.00
[Mobile App]  Alert: Withdrawal of $15000.00 failed (Insufficient Funds).
[Analytics]  Metric recorded: Event [FailedWithdrawal] for $15000.00

=== PATTERN DEMO COMPLETED ===
========================================
```

Figure 4: Interactive Console Output of the Observer Pattern Bank Account System

8 Pros and Cons

8.1 Pros

- **Loose Coupling:** The Subject and Observers are loosely coupled. The Subject only knows that the observer implements a specific interface.
- **Support for Broadcast Communication:** Unlike direct calls, notifications are broadcast automatically to all registered subscribers.
- **Open-Closed Principle:** We can introduce new concrete observer classes without changing the subject's code.

8.2 Cons

- **Lapsed Listener Problem:** Observers must be carefully deregistered when destroyed, or they remain in memory and get called, causing undefined behavior or leaks.

- **Unintended Update Chains:** If observers trigger updates on subjects, it can lead to complex and hard-to-debug cascading updates or infinite loops.
- **Ordering Concerns:** There is no guarantee in which order the observers will be notified.

9 Other Real-world Applications

9.1 Web Development

- **Event Handling in JavaScript:** The DOM's `addEventListener` uses the Observer pattern to attach event handlers to HTML elements.
- **State Management (Redux/Vuex):** Views subscribe to changes in a global store and re-render automatically when state mutations occur.

9.2 Mobile Development

- **RxSwift / RxJava / RxKotlin:** Reactive streams rely heavily on observing data sequences and binding them directly to UI components.
- **SwiftUI/Android Jetpack Compose:** Declarative UI frameworks use property wrappers like `@Published` and state flows to push model updates directly to the view.

10 Quiz

Here are some interactive questions to review the Observer pattern:

1. What type of design pattern is the Observer Pattern?

   A) Creational
   B) Structural
   C) Behavioral
   D) Architectural

2. Which SOLID principle does the Observer pattern primarily help satisfy when adding new subscribers?

   A) Liskov Substitution Principle
   B) Single Responsibility Principle
   C) Interface Segregation Principle
   D) Open-Closed Principle

3. What is the "Lapsed Listener" problem in the Observer pattern?

   A) The subject stops listening to updates.
   B) Observers are not notified because the subject crashed.

C) An observer is deleted without unsubscribing, causing memory leaks/crashes.

D) Multiple subjects occupy the same listener list.

4. Which of the following is a key advantage of the Observer pattern?

   A) It ensures observers are updated sequentially.

   B) It decouples the subject from concrete observer details.

   C) It reduces memory overhead of notifications.

   D) It prevents subjects from changing their state.

Answer Key: 1-C, 2-D, 3-C, 4-B.

11 References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Refactoring.Guru. *Design Patterns: Observer.* https://refactoring.guru/design-patterns/observer